

Routing using Delegation Mechanisms Inspired from Liquid Democracy

Aditya Vema Reddy Kesari
IIT Bombay
Mumbai, India

Shresth Verma
IIT Bombay
Mumbai, India

Kanishk Sharma
IIT Bombay
Mumbai, India

ABSTRACT

We address the problem of routing queries across a network of deep neural network (DNN) classifiers trained on diverse tasks, with the aim of identifying the most suitable classifier to handle any given input. Instead of relying on reinforcement learning, we take inspiration from *liquid democracy*, a flexible delegation paradigm studied in the context of collective decision-making—and propose a novel delegation-based approach to routing.

In our framework, each DNN node has a prior over the expected performance on a given input image, both for itself and its neighboring nodes, based on uncertainty estimates derived from its penultimate layer. We analyze model behavior on actual classification examples and simulate uncertainty scores using a parametric function to evaluate delegation-based routing. Our results show that such delegation strategies can effectively route queries to more confident models, offering a transparent and efficient alternative to learned routing policies.

This approach is especially useful in scenarios where full access to a model is not possible or desirable. For example, a large and sensitive model, such as a proprietary language model, can be partitioned into smaller components hosted across a distributed network. A user can query one of the outer nodes in the network, and based on uncertainty, the system can delegate the query internally to a more confident node, without revealing or exposing the entire model. This enables secure, efficient, and privacy-aware deployment of powerful models in distributed environments.

KEYWORDS

Model routing, Deep neural networks, Uncertainty estimation, Liquid democracy, Delegation strategies, Distributed inference, Query delegation, Multi agent systems

1 INTRODUCTION

In recent years, deep neural networks (DNNs) have achieved state-of-the-art performance across a variety of tasks, including image classification, natural language processing, and reinforcement learning. However, deploying these models in real-world applications introduces several challenges, particularly when multiple DNN classifiers are involved in solving related tasks in a distributed setting. One key issue is efficiently routing queries to the most suitable model, ensuring high accuracy and timely responses while minimizing the computational cost.

Traditional approaches to model routing, such as reinforcement learning (RL) based strategies, often rely on learning complex decision policies that optimize query handling over time. These methods typically require extensive training data and can be computationally expensive, especially in large-scale, dynamic environments where multiple classifiers are distributed across nodes.

In this paper, we introduce a novel framework for routing queries across a network of DNN classifiers by taking inspiration from *liquid democracy*, a flexible delegation model widely studied in collective decision-making systems. In liquid democracy, decisions can be delegated to trusted individuals or nodes, allowing for efficient decision-making without requiring direct involvement of all participants. We adapt this concept to the context of model routing, where each DNN node has the ability to delegate queries to neighboring models based on uncertainty metrics, ensuring that high-confidence models handle the queries.

The central idea of our framework is to leverage each node’s uncertainty about its predictions, which can be inferred from the model’s penultimate layer, as a basis for routing decisions. Each node maintains a prior over its expected performance on a given input and also performance of its neighboring nodes. The delegation process is based on confidence scores assigned to each model and its neighbors. This approach draws inspiration from the principles of liquid democracy, where nodes “delegate” the decision-making process to better models when necessary, ensuring that the query is resolved by the most capable model.

One of the key advantages of this delegation-based approach is its applicability to scenarios where full model sharing is not possible or desirable. For instance, in cases involving large, proprietary models—such as large language models or specialized image recognition networks—full access to the model may be restricted for privacy, security, or intellectual property reasons. Our framework allows such models to be partitioned across a distributed network, where users can query outer nodes without gaining access to the entire model. The query is then routed to the most suitable model in the network based on uncertainty, maintaining both model integrity and high-quality service.

The rest of the paper is organized as follows. In Section related work, we review related work in model routing, uncertainty estimation, and delegation-based systems. In Section methodology, we describe the details of our framework. In Section experiments, we present experimental results demonstrating the effectiveness of our approach using real-world datasets such as MNIST. Finally, we conclude with a discussion of the implications of our findings and potential directions for future work.

2 RELATED WORK

Liquid democracy is a modern voting paradigm that combines the strengths of both direct and representative democracy. Unlike traditional proxy voting (Miller, 1969), where a voter assigns their vote to another individual only once, liquid democracy supports *transitive delegation*—votes can be passed through a chain of delegates in a *delegation graph* until they reach a final decision-maker. This

model has gained practical relevance, having been adopted by civic groups such as the German Pirate Party and Sweden's Demoex.

The core motivation behind liquid democracy lies in addressing a key limitation of direct democracy: not all voters are equally informed or willing to participate in every decision. As discussed by Green-Armytage (2015), direct democracy can be inefficient due to voter apathy or lack of expertise. Liquid democracy allows less informed individuals to delegate their vote to someone more knowledgeable, thereby improving the overall quality of collective decisions.

Recent work has focused on the algorithmic underpinnings of delegation mechanisms, particularly in the context of improving decision accuracy. One such area is *local delegation*, where voters independently choose from a small set of trusted neighbors to delegate their votes. While intuitive, local delegation can lead to disproportionate concentration of votes among a few individuals. If these individuals are less competent or make mistakes, the entire decision process suffers.

Kahng, Mackenzie, and Procaccia [3] examine this challenge and formalize two desirable properties for any delegation mechanism:

- **Positive Gain (PG):** Delegation should increase the probability of a correct collective decision.
- **Do No Harm (DNH):** Delegation should not degrade decision quality compared to direct voting.

They show that local delegation mechanisms frequently fail to satisfy both properties. To address this, the authors propose a novel non-local mechanism called **GreedyCap**. GreedyCap works by iteratively identifying individuals who have received the most approvals and delegating votes to them, while enforcing a cap on how many votes any individual can accumulate. This avoids excessive concentration of power and ensures a more balanced distribution of influence.

GreedyCap is proven to satisfy the PG property, especially in large-scale networks. Although it does not always satisfy DNH in small graphs, it performs well when voters have a competence level bounded away from 0 and 1. The authors present a compelling example in a star-shaped graph where local delegation routes all votes to a central node, risking incorrect outcomes if that node is unreliable. GreedyCap avoids this pitfall by distributing votes more evenly, leading to a more accurate collective decision.

This illustrates that careful design of delegation mechanisms, especially non-local ones like GreedyCap—can significantly enhance the reliability and fairness of decision-making in liquid democracy systems. An example in the paper shows how local delegation fails, but GreedyCap succeeds, especially in a star-shaped graph. In this scenario, local delegation leads to a decision that depends entirely on the central voter's competence, risking errors. GreedyCap avoids this by preventing all votes from being concentrated on the central voter, leading to more accurate decisions. The diagram below illustrates this scenario.

Drawing inspiration from liquid democracy for a different machine learning challenge, Armstrong and Larson (2024) [1] address the high computational cost associated with *training* large classifier ensembles. They propose an incremental training and *ensemble*

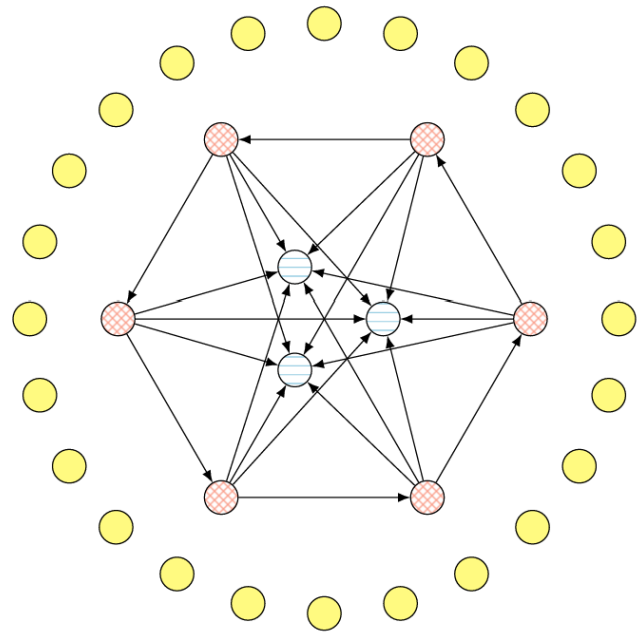


Figure 1: Star Graph: Failure of Local Delegation

pruning procedure where delegation mechanisms are used to identify and effectively remove redundant or underperforming classifiers. In their framework, an ensemble is trained over sequential batches of data. After each training increment, a subset of classifiers, typically those exhibiting the lowest average accuracy, are designated as delegators. These delegators then transfer their associated weight to other, higher-performing classifiers remaining in the ensemble.

Armstrong and Larson investigate several delegation probability functions to determine *how* this weight transfer occurs, including mechanisms like delegating to the single highest-accuracy model, randomly among better models, or proportionally based on accuracy differences and existing weights. A key finding is that careful selection of the delegation mechanism, such as their Proportional Weighted approach, is crucial for avoiding excessive weight centralization onto a single classifier and can lead to ensembles that achieve high accuracy (sometimes exceeding Adaboost baselines) while dramatically reducing the overall training cost compared to training the full, unpruned ensemble.

Their work successfully adapts liquid democracy's delegation concept to optimize the *static structure* of an ensemble based on performance aggregated during training. The resulting pruned ensemble is then used for inference. This contrasts sharply with our objective: we leverage liquid democracy principles not for pruning during training, but for dynamic, *instance-specific routing* at *inference time*. Our system uses priors to decide whether to handle the query directly or delegate it to a neighbor perceived as more confident for that particular instance. Therefore, while both approaches utilize delegation, Armstrong and Larson use it to build a cost-effective static ensemble, whereas we use it to determine the

optimal processing path for individual queries within a potentially fixed network of diverse models.

Further extending the application of liquid democracy within machine learning ensembles, Blair et al. (2024) [2] tackle the challenges of *continual learning* (CL). Their work addresses the problem of dynamically selecting which classifiers within an ensemble should be active for learning and/or prediction when faced with non-stationary data streams, aiming to mitigate catastrophic forgetting without relying on explicit context labels or memory replay mechanisms. Their primary proposed method, *k-Best-Accuracy-Trend* (*k-BAT*), operates during the *training phase*. It evaluates classifiers based on the recent trend (slope) of a chosen performance metric (like accuracy or F1-score) over a sliding window. Based on these trends, the algorithm selects '*k*' "gurus" deemed best suited for the current data distribution, which remain active for learning. Other classifiers then delegate their participation, potentially transitively, using various delegation probability functions inspired by liquid democracy research.

They also introduce a "Student-Expert" mechanism that extends delegation principles to *test time*, allowing the ensemble to adapt its predictions based on performance observed on an ordered test stream. The core idea in both mechanisms is to use aggregated performance trends over recent batches as a proxy for model suitability in the current (potentially unknown) context. This focus on leveraging *historical performance dynamics* for adapting the ensemble's learning or testing behavior in a CL setting contrasts with our approach. Our framework operates purely at *inference time* for individual query routing, making delegation decisions based on *input-specific uncertainty estimates* derived from model internals for the current query, rather than relying on performance trends aggregated over past data batches. While Blair et al. use delegation to manage learning and adaptation over time, we use it to find the best single-instance processor within a potentially static network.

3 METHOD

We use the CIFAR-100 dataset, which consists of 100 fine-grained classes grouped into 20 coarse categories. Each fine class corresponds to a specific object or concept, such as "apple" or "car," while the coarse categories group similar fine classes, such as "fruit and vegetables" or "vehicles." For our experiments, we employ a set of 40 models, each trained on a random selection of two or three fine classes from the CIFAR-100 dataset. This diverse ensemble of models allows us to cover a broad range of fine-grained categories, ensuring variability and robustness in the evaluation of our routing mechanism.

In the interest of time and resources, instead of training one model per classification model for developing priors, instead, based on the confusion matrix of a model trained on all classes of CIFAR 100, we model uncertainty in predictions. Confidence scores are assigned based on the relationship between the input image's true class and the classes each model is trained on. A score of 1.0 is given for an exact match, 0.35 for the same coarse class, and 0.11 for similar coarse classes. These scores are modeled as Gaussian distributions with means of 1.0, 0.35, and 0.11, respectively, and standard deviations of 0.1, 0.03, and 0.01, reflecting the varying levels of uncertainty. The vote propagation process begins with the

query model (the starting point) receiving 10 votes, while all other models begin with 0 votes. Each model delegates votes iteratively to its neighbors according to the proportional ratio of their confidence scores, inspired by the proportional ratio-based method in liquid democracy. This means that models with higher confidence in the prediction will receive a greater share of the votes from their neighbors. The propagation continues iteratively, with votes being redistributed in each step, until the votes assigned to any model fall below a predefined threshold, at which point the propagation halts. This ensures that only models with sufficiently strong confidence influence the final decision.

4 EXPERIMENTS

In our experiments, we evaluate the performance of the vote propagation system using ensembles of models trained on different subsets of the CIFAR-100 dataset, which includes 100 fine-grained classes grouped into 20 coarse categories. We vary both the number of models in the ensemble and the number of neighbors each model can delegate votes to. For each configuration, the system is tested across multiple random query starts, and the final prediction is determined by the model that accumulates the most votes at the end of the propagation process.

We report the following evaluation metrics: accuracy, precision, recall, top-3 hit rate, and average number of models queried. Accuracy indicates the proportion of correct top-1 predictions. Precision and recall measure the correctness and completeness of positive predictions, respectively, averaged over all test cases. The top-3 hit rate measures how often the correct class appears among the top three voted predictions, providing insight into the system's broader predictive relevance. Lastly, the average number of models queried quantifies computational cost by reporting how many models received non-zero votes during propagation. Standard deviations across multiple trials are included to assess result stability.

4.1 Effect of Number of Models on Ensemble Performance

To analyze the effect of the number of models and neighbours used in our vote propagation ensemble, we conducted experiments by varying the number of base models and their corresponding neighbours. The evaluation metrics reported include Accuracy, Precision, Recall, Top-3 Hit Rate, and the Average Number of Models Queried.

For each image, the start model is selected uniformly at random. To account for the randomness introduced by this process, the ensemble inference is run 5 times per image. All reported metrics are therefore averaged over these 5 runs, and we report both the mean and standard deviation to capture the variability introduced by random starts.

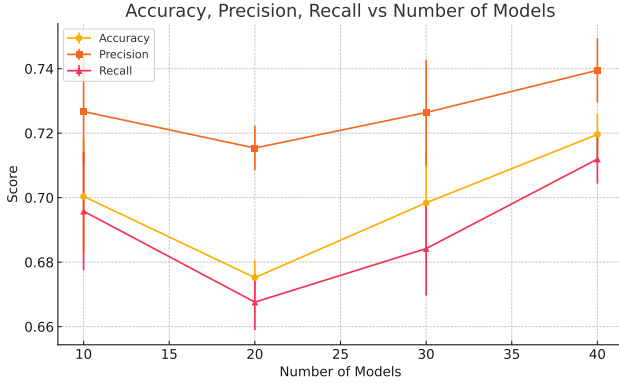


Figure 2: Accuracy, Precision, and Recall vs Number of Models (mean $\pm$ std over 5 runs)

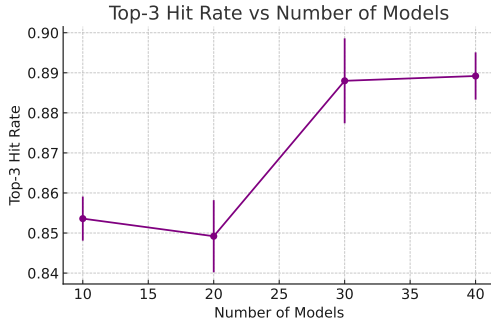


Figure 3: Top-3 Hit Rate vs Number of Models (mean $\pm$ std over 5 runs)

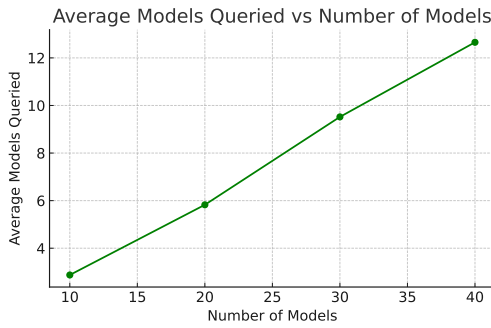


Figure 4: Average Models Queried vs Number of Models (mean $\pm$ std over 5 runs)

Observations and Inference. From Figure 2, we observe that increasing the number of models generally improves all three metrics: accuracy, precision, and recall. While performance dips slightly at 20 models, it recovers and peaks at 40 models, with relatively small standard deviations across all values. This indicates increased reliability and stability with larger ensembles.

Figure 3 shows a similar trend for the Top-3 Hit Rate. While the values remain relatively high across all settings, a noticeable jump occurs when the ensemble grows beyond 20 models. This suggests that diversity among models contributes to better class coverage in the top predictions.

In Figure 4, we see that the average number of models queried increases almost linearly with the total number of models in the ensemble. Although this leads to a higher computational cost, the trade-off is justified given the consistent improvement in classification performance. We aimed for sub linear growth in number of queries which we failed to achieve.

Conclusion. These results demonstrate that using a larger number of models in the ensemble, coupled with a proportionally increasing number of neighbours in the vote propagation system, can significantly enhance performance. However, this comes at the cost of increased model queries, which must be managed based on resource constraints.

4.2 Effect of Model-to-Neighbor Ratio on Propagation Performance

To better understand the effect of graph sparsity on the label propagation process, we fix the number of neighbors to 4 and vary the total number of models in the system. This allows us to control the model-to-neighbor ratio, which governs the density of the underlying propagation graph. A higher ratio corresponds to a sparser graph. As before, each query image is evaluated five times with randomly selected starting models, and the mean and standard deviation across runs are reported.

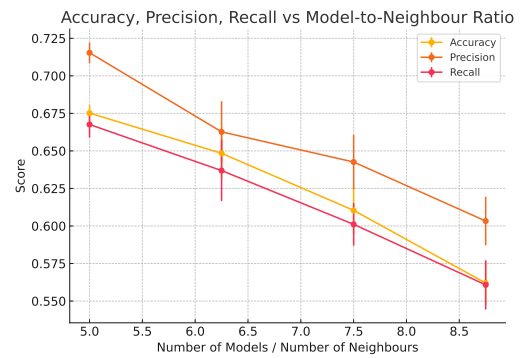


Figure 5: Accuracy, Precision, and Recall vs. Model-to-Neighbor Ratio. Mean and standard deviation over 5 runs are shown.

Inference: As the model-to-neighbor ratio increases, accuracy, precision, and recall all decline. This suggests that denser connectivity (lower ratio) allows more effective aggregation of confidence

from neighboring models, leading to improved predictions. In contrast, sparsity limits the ability to reinforce correct labels during propagation.

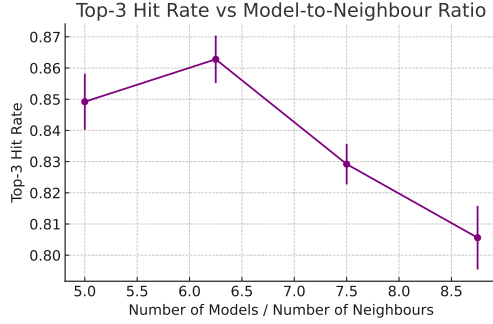


Figure 6: Top-3 Hit Rate vs. Model-to-Neighbor Ratio. Mean and standard deviation over 5 runs are shown.

Inference: The top-3 hit rate also decreases with increasing model-to-neighbor ratio. This reflects that denser graphs support better exploration.

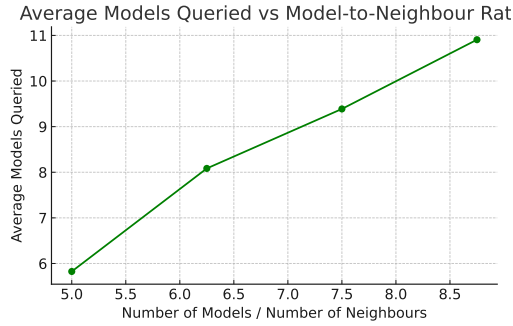


Figure 7: Average Models Queried vs. Model-to-Neighbor Ratio. Mean and standard deviation over 5 runs are shown.

Inference: The average number of models queried grows as the graph becomes sparser. This is because propagation must traverse more nodes to accumulate enough vote weight for prediction, increasing computational cost and potentially reducing confidence due to longer chains.

5 CONCLUSION

We present a novel formulation for routing classification queries across a network of distributed deep learning models using a liquid democracy-inspired vote propagation mechanism. This framework enables individual models to express uncertainty and delegate decisions to neighboring models with higher confidence. Our results demonstrate consistently high accuracy and top-3 hit rates, indicating the effectiveness of the voting-based routing system in leveraging collective expertise. Furthermore, we analyze key trade-offs between accuracy and model querying cost under varying configurations, highlighting the system’s flexibility. We have essentially

explored local delegation mechanisms in these experiments and obtained similar dissatisfactory results as the number of models queried does not grow sub linearly. A future direction is to perhaps explore well thought out global mechanisms to achieve this.

REFERENCES

- [1] Ben Armstrong and Kate Larson. 2024. Liquid Democracy for Ensemble Pruning in Deep Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. <https://arxiv.org/abs/2402.12345> Preprint.
- [2] Carter Blair, Ben Armstrong, and Kate Larson. 2024. Liquid Ensemble Selection for Continual Learning. In *Proceedings of the 37th Canadian Conference on Artificial Intelligence (Canadian AI 2024)*. <https://assets.pubpub.org/hyoynx2h/Blair-31716794237544.pdf>
- [3] Alexander Kahng, Simon Mackenzie, and Ariel D. Procaccia. 2021. Liquid Democracy: An Algorithmic Perspective. In *Proceedings of the 1st Workshop on Gamification and Incentivization (GAIW 2021)*. https://preflib.github.io/gaiw2021/papers/GAIW_2021_paper_2.pdf